

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Introduction

Introduction

There is a connection between the Tsai-Li framework¹ and Software Transactional Memory (STM) mechanisms

¹J. J. Tsai and H. C. Li, "Lock-free concurrent tree structures for multiprocessor systems (extended abstract),"

The first framework introduces the behavior of concurrent binary search trees, while the second stands on a generalized approach to executing transactions as inter-process communication mechanisms in shared memory. Their connection lies in their shared ability to deliver non-blocking properties to complex systemsCAS instruction atomically replaces one value with another. Although restricted to a single memory location, its consensus number is infinite, meaning it can be utilized to implement any non-blocking interactionThis tool specifies the Temporal Logic of Actions, enabling the definition of relations between the system's previous and next states. A specification within this framework consists of an initial state and a next-state predicate

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Introduction

Introduction

There is a connection between the Tsai-Li framework¹ and Software Transactional Memory (STM) mechanisms

First, both use Multiple Compare-And-Swap (or analogy), where

$$\text{CAS}(\text{value}, \text{old}, \text{new}) \triangleq \vee (\text{value} = \text{old} \wedge \text{value}' = \text{new})$$

$$\vee (\text{value} \neq \text{old} \wedge \text{value}' = \text{value}) \text{ atomically}$$

¹J. J. Tsai and H. C. Li, "Lock-free concurrent tree structures for multiprocessor systems (extended abstract)"

The first framework introduces the behavior of concurrent binary search trees, while the second stands on a generalized approach to executing transactions as inter-process communication mechanisms in shared memory. Their connection lies in their shared ability to deliver non-blocking properties to complex systems

CAS instruction atomically replaces one value with another. Although restricted to a single memory location, its consensus number is infinite, meaning it can be utilized to implement any non-blocking interaction

This tool specifies the Temporal Logic of Actions, enabling the definition of relations between the system's previous and next states. A specification within this framework consists of an initial state and a next-state predicate

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Introduction

Introduction

There is a connection between the Tsai-Li framework¹ and Software Transactional Memory (STM) mechanisms. First, both use Multiple Compare-And-Swap (or analogy), where

$$\text{CAS}(\text{value}, \text{old}, \text{new}) \triangleq \vee (\text{value} = \text{old} \wedge \text{value}' = \text{new})$$

$$\vee (\text{value} \neq \text{old} \wedge \text{value}' = \text{value}) \text{ atomically}$$

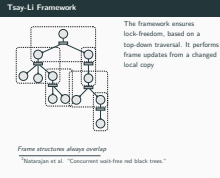
Second, we can establish this connection by utilizing TLA+ as a specification tool. For instance, the model can be verified using the TLC model checker

¹J. J. Tsai and H. C. Li, "Lock-free concurrent tree structures for multiprocessor systems (extended abstract)"

The first framework introduces the behavior of concurrent binary search trees, while the second stands on a generalized approach to executing transactions as inter-process communication mechanisms in shared memory. Their connection lies in their shared ability to deliver non-blocking properties to complex systems. CAS instruction atomically replaces one value with another. Although restricted to a single memory location, its consensus number is infinite, meaning it can be utilized to implement any non-blocking interaction. This tool specifies the Temporal Logic of Actions, enabling the definition of relations between the system's previous and next states. A specification within this framework consists of an initial state and a next-state predicate.

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

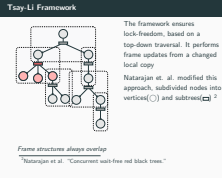
└─Tsay-Li Framework



All operations modifying the frame copy it locally before executing a full state update. This approach is poorly suited for CAS, as it requires the simultaneous modification of multiple variables. They adapted the framework for modern processors by decoupling vertices and edges into two distinct entities. Therefore, it is now possible to update only the root, effectively discarding the previous frame's state from history. To address this, they proposed employing descriptors to store comprehensive operation metadata. I wrote specification to verify correctness and lock-freedom of this framework. This spec will be used later for build Multiple CAS as a generalized algorithm, based on previous specification.

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

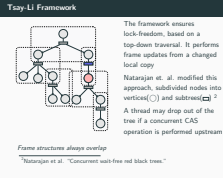
└─Tsay-Li Framework



All operations modifying the frame copy it locally before executing a full state update. This approach is poorly suited for CAS, as it requires the simultaneous modification of multiple variables. They adapted the framework for modern processors by decoupling vertices and edges into two distinct entities. Therefore, it is now possible to update only the root, effectively discarding the previous frame's state from history. To address this, they proposed employing descriptors to store comprehensive operation metadata. I wrote specification for verify correctness and lock-freedom of this framework. This spec will be used later for build Multiple CAS as a generalized algorithm, based on previous specification.

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

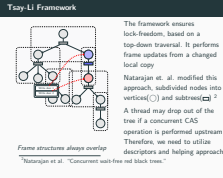
└ Tsay-Li Framework



All operations modifying the frame copy it locally before executing a full state update. This approach is poorly suited for CAS, as it requires the simultaneous modification of multiple variables. They adapted the framework for modern processors by decoupling vertices and edges into two distinct entities. Therefore, it is now possible to update only the root, effectively discarding the previous frame's state from history. To address this, they proposed employing descriptors to store comprehensive operation metadata. I wrote specification to verify correctness and lock-freedom of this framework. This spec will be used later for build Multiple CAS as a generalized algorithm, based on previous specification.

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Tsay-Li Framework



All operations modifying the frame copy it locally before executing a full state update. This approach is poorly suited for CAS, as it requires the simultaneous modification of multiple variables. They adapted the framework for modern processors by decoupling vertices and edges into two distinct entities. Therefore, it is now possible to update only the root, effectively discarding the previous frame's state from history. To address this, they proposed employing descriptors to store comprehensive operation metadata. I wrote specification for verify correctness and lock-freedom of this framework. This spec will be used later for build Multiple CAS as a generalized algorithm, based on previous specification.

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Adaptation

Modifying a frame now required marking all participating vertices via CAS, executing an atomic CAS replacement, and subsequently releasing them for subsequent operations. This means that it is built upon operations whose unique sequence strictly determines from the state of the descriptor

Adaptation

To transition from trees to flat sets, values must be moved from nodes directly into descriptors. It should consider three phases:

- **Acquire:** Thread installs its descriptor D into target cells R_j via CAS in ascending address order (prevents deadlocks). If blocked by descriptor U , the thread helps U finish first.

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Adaptation

Modifying a frame now required marking all participating vertices via CAS, executing an atomic CAS replacement, and subsequently releasing them for subsequent operations. This means that it is built upon operations whose unique sequence strictly determines from the state of the descriptor

Adaptation

To transition from trees to flat sets, values must be moved from nodes directly into descriptors. It should consider three phases:

- **Acquire:** Thread installs its descriptor D into target cells R_j via CAS in ascending address order (prevents deadlocks). If blocked by descriptor U , the thread helps U finish first.
- **Decision (Linearization Point):** Once all cells are acquired, the thread atomically changes descriptor status via CAS from *Undecided* to *Committed* or *Aborted*.

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Adaptation

Modifying a frame now required marking all participating vertices via CAS, executing an atomic CAS replacement, and subsequently releasing them for subsequent operations. This means that it is built upon operations whose unique sequence strictly determines from the state of the descriptor

Adaptation

To transition from trees to flat sets, values must be moved from nodes directly into descriptors. It should consider three phases:

- **Acquire:** Thread installs its descriptor D into target cells R_j via CAS in ascending address order (prevents deadlocks). If blocked by descriptor U , the thread helps U finish first.
- **Decision (Linearization Point):** Once all cells are acquired, the thread atomically changes descriptor status via CAS from *Undecided* to *Committed* or *Aborted*.
- **Release:** Thread resets cell states to concrete values. Updates to *new_j* if *Committed*, or reverts to *old_j* if *Aborted*, exposing the changes to the system.

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Adaptation

Modifying a frame now required marking all participating vertices via CAS, executing an atomic CAS replacement, and subsequently releasing them for subsequent operations. This means that it is built upon operations whose unique sequence strictly determines from the state of the descriptor

Adaptation

To transition from trees to flat sets, values must be moved from nodes directly into descriptors. It should consider three phases:

- **Acquire:** Thread installs its descriptor D into target cells R_j via CAS in ascending address order (prevents deadlocks). If blocked by descriptor U , the thread helps U finish first.
- **Decision (Linearization Point):** Once all cells are acquired, the thread atomically changes descriptor status via CAS from *Undecided* to *Committed* or *Aborted*.
- **Release:** Thread resets cell states to concrete values. Updates to *new_j* if *Committed*, or reverts to *old_j* if *Aborted*, exposing the changes to the system.

Since the specification draws no difference between threads and their descriptors, all updates occur concurrently.

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Algorithm Specifications

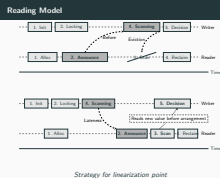
State variables:

- `vdescState` Descriptor status (*IDLE, ACTIVE, SCANNING, SUCCESS, ABORTED*).
- `vdescPending` Cells awaiting locking or physical update.
- `vdescFrame` Keys (cells) acquired by the transaction.
- `vdescOldValue` Snapshot of original cell values (*old_j*).
- `vdescNewValue` Snapshot of target cell values (*new_j*).

The specification uses the following temporal variables that change over time based on the `Next` predicate. This predicate is built from a set of conditions that refine the behavior of phases, whose described on previous slide. Executing operations requires helping other threads, which degrades read-only transaction performance. Therefore, readers simply announce their presence, and writers track these announcements

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Reading Model



Here we see two main behaviors of this system. In the first case, the writing thread aborts the read operation since it changed the state. We don't care about the specific implementation; what matters is that it can be done efficiently using CAS. In the second scenario, the writer is late, shifting linearizability careness to the reader. Thus, reading before the Decision phase yields the old value; otherwise, it returns the new one, as if already updated in memory.

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Evaluation

Evaluation

- **Configuration:** 2 writing threads, 1 reading thread, and a 2-call memory pool.
- **State Space Constraint:** Increasing the number of threads causes an exponential state-space explosion.

Here is why I find this configuration sufficient. While more broader configurations also work, these are the ones currently set in the module configs. Consider an analogy with segmented geometric rays: we only need to check if any two sections intersect, ignoring how many intersections occur elsewhere.

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

└ Evaluation

Evaluation

- **Configuration:** 2 writing threads, 1 reading thread, and a 2-call memory pool.
- **State Space Constraint:** Increasing the number of threads causes an exponential state-space explosion.
- **Sufficiency:** 2 concurrent threads are sufficient because all method phases overlap pairwise. A 3rd thread adds no new interleaving patterns, only duplicating existing overlapping scenarios.
- **Memory Pool:** A 2-call memory pool is enough to verify how the framework handles overlapping frame sections.

Here is why I find this configuration sufficient. While more broader configurations also work, these are the ones currently set in the module configs. Consider an analogy with segmented geometric rays: we only need to check if any two sections intersect, ignoring how many intersections occur elsewhere.